

CSCE 45203/55203

Database Management Systems

Spring 2026

Objective

The goal of this project is to practice creating a relational database backend with a web front end using any technologies you prefer.

You may work to help each other with any web programming issues, but acknowledge any help given in the report. You may implement the application described below, or **choose to create your own database application of similar complexity (except previous 45203/55203 class projects)**.

Requirements

DBMS: may be mysql on turing or any other **relational database** package on any other machine that is accessible 24/7 over the web

Database: may be the database described below or any other database consisting of at least 3 tables

Web server: may be Apache running on turing, or any other web server (AWS?) that is publicly accessible 24/7

Students, including Grad Students, may work in pairs. However, each member of the pair needs to have their own copy of all the code and therefore a separate url.

Note: If you and your partner have to pay for the Web account, you may share one installation to avoid paying twice, but you need two different sets of web pages.

Web site: may be the web site described below, or any other web site that supports at least 7 functions, each of which is achieved through a database call.

10 point BONUS: Add a 9th, complex function

Programming Language: may be any programming language. Examples are provided on the class website in several languages called from php. You do not have to use this code, any languages are fine to use.

Program Description

Create a new database to keep track of **music concerts and ticket sales**. There should be at least four tables:

Artist: ArtistId, ArtistName, Genre

Concert: ConcertId, VenueName, City, ConcertDate, ArtistId

Customer: CustomerId, CustomerName

Ticket: TicketId, ConcertId, CustomerId, SeatNumber, Price

You may create different tables or change the attributes around as you see fit. This is just a sample, simple database design that is sufficient for full credit. There must be at least **8 functions** that use the database, at least **3 of which modify the table contents in some way** (i.e., add, insert, delete, or modify) and at least **two of which combine information from multiple tables** to produce the resulting web page, and at least **one of which uses an aggregation** operation.

Create a web interface that supports each of the following functions:

1) Add an artist to the Artist table

(needs join: No; needs aggregation: No)

2) Add a concert to the Concert table

(needs join: No; needs aggregation: No)

3) Add a customer to the Customer table

(needs join: No; needs aggregation: No)

4) Add a ticket purchase to the Ticket table

(needs join: No; needs aggregation: No)

5) View all concerts and/or all concerts for a given City

(For full credit, the user should have an easy way to know which cities are valid, e.g., a dropdown list.

(needs join: No; needs aggregation: No)

6) View all concerts for a given artist

Display the artist name, venue name, city and concert date for each concert.

(needs join: Yes; needs aggregation: No)

7) View total spending per customer

Display the customer id and customer name and the total amount spent on tickets.

It should be possible to view all customers or filter by a specific customer.

(needs join: Yes; needs aggregation: Yes)

8) Find the top 3 artists whose concerts generated the highest total ticket revenue.

Display:

- ArtistName
- TotalRevenue (sum of ticket prices across all their concerts)

(needs join: Yes; needs aggregation: Yes)

BONUS: should require joining multiple tables **and** extending the schema to include more attributes and/or tables

You must also create a single home page that has links to each of the functions for your website. Each result page for the 8 functions above should have an easy way to navigate back to the home page, if necessary (without using the "Back" button").

The website should have a professional look and feel to receive full credit. Any debug statements used during development should be removed from the final version and the web pages should be attractive.

Implementation

Create the tables on the command line or by calling a main program from the server's command line. Seed each table with at least 2 records. Create and test one function from the server's command line, then create a web page that calls it. Only after a single function is completed should you move on and use that function/page as a template for the remaining programs.

General Directions

Implement the program in a professional way, i.e., comment it, use classes and/or functions, white space, mnemonic variable names

Implement the functions one at a time and test them thoroughly. You will receive more credit for a subset of the functions that work correctly than for code which attempts all functions but is poorly implemented.

Create a short report that includes screenshots of the homepage and, for each of the 8 (or more) actions, the web page and one result page. In the report, discuss: the url for the project home page, which language(s) you used, what host server you chose, what DBMS, and your database design (tables, attributes, datatypes, constraints). Also discuss any error checking you did.

Use zip to put the files all together to name it myIDNum.zip and submit it via the homework submission link on Blackboard on or before the due date.

Grading: Correctness (80 points; 10 points for each of the 8 functions), Web design (12 points), Report (8 points), Code Design (10 points), Database Design (20 points)

Where to work: Wherever you like.

What to submit: Submit one zipfile *per pair*. If you worked in pairs, be sure that the report lists both names and includes both urls. The other partner should upload a short report that just says *I worked with NAME*.